CISC3010 - Distributed Systems

Pedro Castelo de Souza Nemetz - UC594395

Flink Event Time Stream Processing

Topics

1. Introduction
2. System Architecture
3. Implementation
4. Experiment
5. Limitation
6. Conclusion
7. References

Introduction

In modern distributed systems, data is often produced continuously over time. However, processing these continuous streams in real-time introduces a fundamental challenge: network delays and out-of-order data arrivals.

My project explores the difference between Processing Time and Event Time. The core question of my project was: how to guarantee correct and consistent results when events arrive late or out of order? To solve this, Apache Flink uses a mechanism called Watermarks, which tracks the progress of Event Time.

System Architecture

To address this problem, I implemented the architecture proposed by the original Apache Flink paper (IEEE 2015) on my project. Flink unifies stream and batch processing using a distributed dataflow engine.

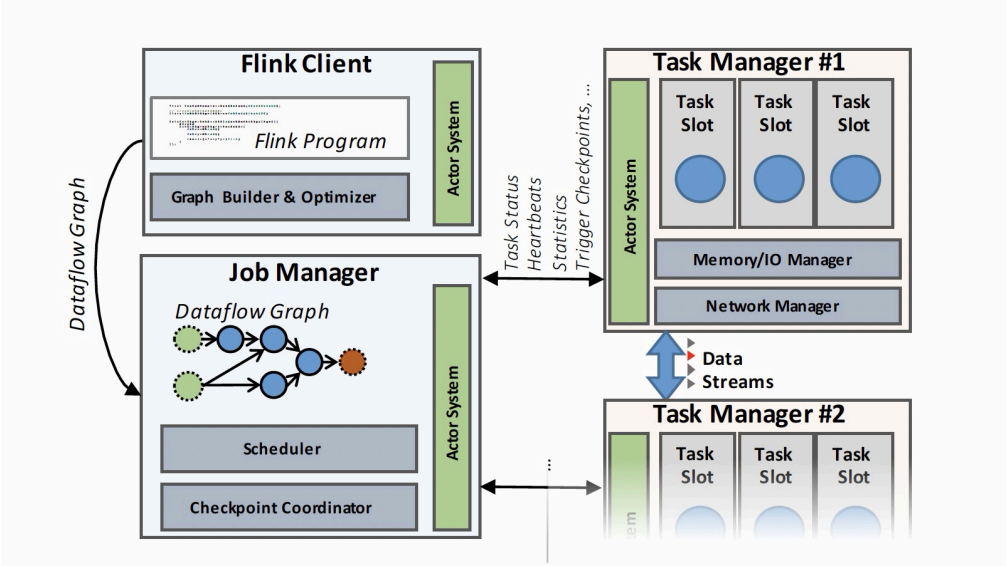
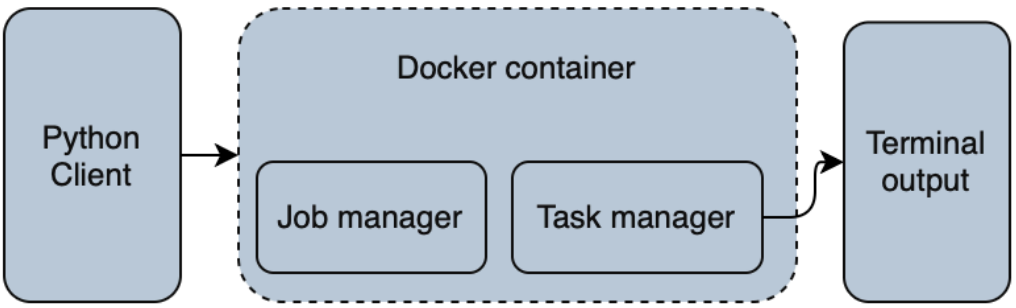


Figure 2: The Flink process model.

The architecture was replicated locally using Docker, consisting of three main components:

Python Client: A PyFlink script that acts as the data source, generating simulated sensor events.
JobManager (Docker): The master node that coordinates the distributed execution.
TaskManager (Docker): The worker node that executes the actual windowing logic and processes the data streams.



Implementation

The implementation was written in Python using PyFlink. I prepared the script to simulate a stream of sensor data containing an ID, an event timestamp, and a temperature value.

To test the system's resilience to out-of-order data, the stream generates on-time events (timestamps 1000, 1500 and 2000) and then injects a late event (timestamp 1200). To force the system to close the time window and evaluate the late event, a "future" event with timestamp 10000 was injected.

The core mechanism is the WatermarkStrategy, configured with for_bounded_out_of_orderness. This allows the system to tolerate a specific amount of delay before closing a window.

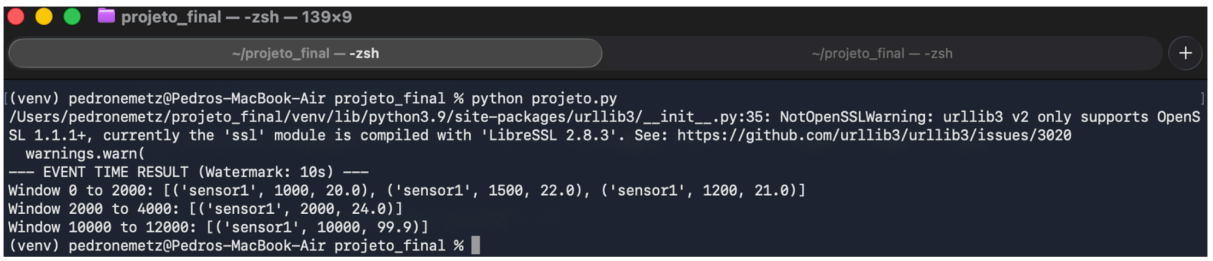
Experiment

The experiment compares two configurations: Watermark of 0 seconds and a Watermark of 10 seconds.

Watermark = 0s
With a 0 second delay, the system does not wait for late events. Because the local processor was really fast, the late event (1200) actually made it into the window. However, in a real network with latency, this configuration would close the window immediately, causing data loss. This proves low latency, but low completeness.



Watermark = 10s
With a 10s delay, the system waited for the Watermark to advance. As a result, the late event (1200) was successfully included in the correct window (0 to 2000). This configuration guarantees high completeness, but at the cost of higher latency.



Limitation

Because the project ran locally on high-performance, the processing speed was faster than the simulated network delay. Because of that the data was not actually discarded in the 0s test, so in this case the theoretical behavior was observed rather than a real network failure.

Conclusion

My experiment demonstrated the fundamental trade-off in stream processing: to guarantee accuracy/completeness, the system must sacrifice speed (latency). Apache Flink provides the tools (Event Time and Watermarks) to handle out-of-order data effectively, but the developer must choose the right balance based on the business requirements.

References

Carbone, P., Katsifodimos, A., Ewen, S., Markl, V., Haridi, S., & Tzoumas, K. (2015). Apache Flink™: Stream and Batch Processing in a Single Engine. IEEE Data Engineering Bulletin.

Apache Flink Documentation: Event Time and Watermarks.

Akidau, T., Begoli, E., Chernyak, S., et al. (2021). Watermarks in Stream Processing Systems: Semantics and Comparative Analysis of Apache Flink and Google Cloud Dataflow. Proceedings of the VLDB Endowment.

I acknowledge the use of DeepSeek (https://chat.deepseek.com) for coding assistance and text structuring. I double verified everything generated and executed the experiment locally.